

Redline Auto Repair

Service Intake Triage Agent — Full Implementation Plan

Purpose of this document: a complete, self-contained build spec you can hand to an AI coding assistant (or follow yourself) to implement UC-0003 end-to-end. It covers the technology stack, project structure, the LangGraph agent design (nodes, tools, edges, state, checkpointing, human-in-the-loop), the Flask backend, a plain HTML/CSS/JS frontend with a configurable model/API panel, the SQLite persistence schema, and a phased build order.

Constraints assumed: everything runs 100% locally (no cloud deployment required), no real money or external systems are touched (all financial actions are simulated/staged in the local database), and the person building this is new to LangGraph, so every concept is explained before it's used.

Stack at a glance: Python 3.11+, Flask (backend + REST API), LangGraph + LangChain (agent orchestration), SQLite (persistence — requests, audit log, LangGraph checkpoints), Pandas (CSV/ledger handling), Pydantic (structured LLM outputs), vanilla HTML/CSS/JS (frontend, no build step, so it looks good instantly and is easy to demo), Chart.js + a small Mermaid/SVG graph view for visualizing the agent.

1. Answering Your Direct Questions First

1.1 Can the frontend be plain HTML/CSS/JS with a Flask + LangGraph backend?

Yes — this is actually the recommended architecture for a competition build. It is fast to build, has zero frontend build tooling (no npm/webpack/React needed, though you may optionally use small CDN libraries like Chart.js), is trivial to run fully offline, and demos cleanly on a judge's laptop with a single `python app.py` command. LangGraph is a Python library — it runs inside your Flask process, not as a separate service. Flask route handlers call into a LangGraph-compiled graph exactly like calling any other Python function/object.

1.2 What is possible locally, and what is not?

Capability	Local feasibility
Running the full LangGraph agent (classification → tools → routing → HITL)	Fully possible, runs on your machine.
Using a hosted LLM (OpenAI, Anthropic, Google, Groq, etc.) via API key	Possible — the app runs locally, only the LLM call leaves your machine over HTTPS.
Using a fully local/offline LLM (no internet, no API key)	Possible via Ollama or LM Studio, both of which expose an OpenAI-compatible <code>/v1/chat/completions</code> endpoint. Your app can point at <code>http://localhost:11434/v1</code> instead of a cloud URL.
Persisting requests, classifications, reasoning traces, overrides	Fully possible with SQLite — no external DB needed.
LangGraph checkpointing / pausing for human approval / resuming later	Fully possible with LangGraph's built-in <code>SQLiteSaver</code> .
Real refunds, real emails, real OEM warranty portal lookups	Not possible/needed and not required by the brief. These are simulated : the 'Courtesy Resolution Tool' writes a staged action to your local DB and a 'customer notification' is just a rendered message shown in the UI, not an actual sent email.
Semantic (embedding-based) duplicate detection fully offline	Possible using a local sentence-embedding model (e.g. <code>sentence-transformers</code> , <code>all-MiniLM</code>) — no API call needed for this part, or you can keep it simple with fuzzy string matching (<code>rapidfuzz</code>) which is enough for a competition and needs zero extra downloads.
Switching the model/provider from the UI without restarting the server	Possible — store the active config in the SQLite DB or a JSON file, read it fresh on every LLM call (details in section 6).

Judging note: because tool calls, reasoning traces and overrides are all logged, the demo story is strong even fully offline — you can show the LLM's chain-of-thought, the exact tool arguments it produced, and a human clicking 'Override' and the graph resuming with corrected data.

1.3 Do I need LangGraph specifically, or could I hand-roll this?

You could hand-roll a state machine with plain Python, but the brief explicitly penalizes 'a single, sequential LLM call' and rewards agentic behavior — LangGraph gives you, for free, exactly the primitives the brief is testing for: a persistent typed state, conditional branching, tool-calling loops, retry-on-error loops, and native interrupt/resume for human-in-the-loop. Using it (and being able to explain it) is itself part of what's being evaluated, per the rationale write-up the brief asks you to submit.

2. Technology Stack

Layer	Technology	Why
Language	Python 3.11+	Required by the brief; also LangGraph's primary language.
Agent orchestration	LangGraph (+ LangChain core)	Cyclic graph engine with native state, conditional edges, tool nodes, checkpointing, and interrupts — matches every 'agentic' requirement in the brief.
LLM access layer	Any OpenAI-compatible client (openai Python SDK, or LangChain's ChatOpenAI) pointed at a configurable base_url	Lets the same code call OpenAI, Groq, OpenRouter, or a local Ollama/LM Studio server just by changing the URL/key/model in the UI — no code changes.
Structured output validation	Pydantic v2	Forces the LLM's classification output into a strict schema (category, confidence, extracted entities) so parsing never breaks.
Backend web framework	Flask 3.x + Flask-CORS	Lightweight, no build step, perfect for a REST API in front of the graph.
Database	SQLite (via Python's built-in sqlite3 or SQLAlchemy)	Zero-config, single-file, fully local — used for the audit table AND as LangGraph's checkpoint store.
Tabular data	Pandas	Loading/filtering service_requests.csv and invoice_verification_records.csv, and the invoice-verification tolerance check.
Fuzzy/duplicate matching	rapidfuzz (or sentence-transformers if you want embeddings)	Duplicate/related-history detection tool.
Frontend	HTML5 + CSS3 + vanilla JavaScript (fetch API)	No build tooling; opens instantly; easy for judges to inspect the source.
Frontend charts	Chart.js (via CDN)	Priority distribution / category breakdown charts on the dashboard.
Agent graph visualization	Static SVG/Mermaid diagram embedded in the UI (see section 8) + a live 'trace' panel that highlights the node the agent is currently in	Satisfies the 'visual representation of nodes and tools' ask without extra dependencies.
Live updates (optional, nice for demo)	Server-Sent Events (SSE) endpoint from Flask	Streams node-by-node progress to the browser as the agent runs, so the UI graph lights up node-by-node.

3. LangGraph From Scratch — Concepts You Need (Since You're New To It)

LangGraph models an application as a **directed graph**. Think of it as a flowchart where each box is a Python function (a **node**), the arrows are **edges** that decide what runs next, and a single shared **state** object flows through the whole thing, getting updated at every box. This is exactly the 'agent, not a single call' behavior the brief demands.

3.1 State

The State is a typed dictionary (a Python TypedDict or a Pydantic model) that represents everything the agent knows about one request as it moves through the graph. Every node reads from it and returns a partial update that LangGraph merges back in. For Redline, the state carries the raw request, the classification result, tool outputs, the reasoning trace, the priority score, the routing decision, and any pending/executed autonomous action.

```
from typing import TypedDict, Optional, List, Dict, Any

class TriageState(TypedDict, total=False):
    request_id: str
    customer_name: str
    customer_tier: str
    submitted_at: str
    body_text: str

    classification_category: Optional[str]
    confidence_score: Optional[float]
    extracted_entities: Dict[str, Any]
    reasoning_trace: List[str] # every reasoning step, appended to, never overwritten

    is_duplicate: Optional[bool]
    duplicate_of_request_id: Optional[str]
    context_notes: Optional[str] # extra history pulled by the context-retrieval tool

    invoice_verified: Optional[bool]
    verified_discrepancy: Optional[float]
    claimed_discrepancy: Optional[float]

    priority_score: Optional[int]
    routed_team: Optional[str]

    pending_action: Optional[Dict[str, Any]] # staged courtesy resolution, awaiting approval
    action_status: Optional[str] # "staged" | "approved" | "rejected" | "overridden" | "executed"
    human_override: Optional[Dict[str, Any]]

    error: Optional[str] # last tool/validation error, used by the retry loop
    retry_count: int
```

3.2 Nodes

A node is just a Python function `def my_node(state: TriageState) -> dict:` that takes the current state and returns the fields it wants to change. LangGraph merges the returned dict into the running state automatically. Some nodes call the LLM; others call plain Python functions ('tools').

3.3 Edges — normal vs. conditional

A normal edge always goes $A \rightarrow B$. A **conditional edge** inspects the state after a node runs and decides which of several next nodes to go to, using a small router function. This is how 'if the classification is uncertain, pull more context' and 'if it's a genuine duplicate, auto-close' are implemented — not with if/else inside one giant function, but

as explicit, inspectable branches in the graph.

3.4 Tools

A tool is a plain Python function with a docstring/schema describing its inputs, decorated with LangChain's `@tool` (or just called directly from a node — for a competition build, calling tool functions directly from inside nodes is simpler and just as valid as binding them to the LLM's native tool-calling; the brief only requires that tool calls happen, not a specific binding style). Tools are how the agent reaches outside the LLM to touch real data: the CSV ledger, the SQLite history table, the fuzzy matcher, etc.

Why tools are needed here specifically

- The LLM cannot reliably do exact arithmetic tolerance checks against a ledger — a Python function (pandas filter + a numeric comparison) can, deterministically, every time.
- The LLM has no memory of yesterday's tickets — a tool that queries SQLite gives it that memory on demand.
- Keeping side effects (writing to the DB, staging a resolution) in tools rather than free-text LLM output means every action is auditable and reproducible — critical for the HITL requirement.

3.5 Checkpointer (persistence + pausing)

LangGraph's checkpointer serializes the entire state to a backing store after every node. For this project, use `langgraph.checkpoint.sqlite.SqliteSaver` pointed at your local `redline.db` file. This is what makes 'pause for three hours until a manager clicks Approve, then resume exactly where it left off' possible with zero extra code — you just call `graph.invoke(None, config={'configurable': {'thread_id': request_id}})` again once the approval arrives, and LangGraph reloads the saved state.

3.6 Interrupts (Human-in-the-Loop)

An interrupt is a point in the graph where execution deliberately stops and waits for external input before continuing. You declare it when compiling the graph: `graph.compile(checkpointer=saver, interrupt_before=['courtesy_resolution_node'])`. When the graph reaches that node, it stops, the state is already saved, and your Flask endpoint returns control to the browser with a 'pending_review' status. Later, a manager's Approve/Reject/Override click hits a different Flask route that updates the state and re-invokes the graph, which resumes exactly at the interrupted node.

4. The Redline Agent Graph — Full Node & Tool Map

Below is the complete graph you should build, with every node mapped directly to a numbered requirement from the UC-0003 brief. Build these as separate functions in one `graph.py` module so the mapping to the brief's numbered list stays obvious to a judge.

#	Node name	Type	What it does	Brief requirement
1	ingest_node	Entry	Loads one row from <code>service_requests.csv</code> (or receives one via API) into <code>TriageState</code> .	Req 1 — ingestion
2	classify_node	LLM + structured output	Calls the LLM with a Pydantic schema; returns <code>category</code> , <code>confidence</code> , <code>extracted_entities</code> . Appends to <code>reasoning_trace</code> .	Req 2 — classification
3	router_confidence	Conditional edge	If <code>confidence_score < 0.80</code> → <code>context_retrieval_node</code> . Else → <code>duplicate_check_node</code> .	Req 3 — uncertain handling
4	context_retrieval_node	Tool node	Queries SQLite for the customer's open/related history; appends <code>context_notes</code> ; loops back to <code>classify_node</code> for re-evaluation.	Req 3
5	duplicate_check_node	Tool node	<code>rapidfuzz/embedding</code> similarity of <code>body_text</code> vs. this customer's tickets from the last 48h in SQLite.	Req 3 & 6
6	router_duplicate	Conditional edge	If <code>is_duplicate</code> → <code>auto_close_node</code> . Else → <code>prioritize_node</code> .	Req 6
7	auto_close_node	Tool node (autonomous, no approval needed)	Marks ticket closed-duplicate in SQLite, drafts an acknowledgment message shown in UI. Terminal node.	Req 6
8	prioritize_node	LLM + rules	Combines LLM-detected urgency keywords with <code>customer_tier</code> multiplier and category bounds into <code>priority_score</code> .	Req 4
9	route_team_node	Deterministic tool	Maps <code>classification_category</code> → <code>routed_team</code> via the fixed matrix in section 4.1 below.	Req 5
10	router_invoice_dispute	Conditional edge	If <code>category == 'Invoice & Billing Dispute'</code> AND a dollar figure was extracted → <code>invoice_verification_node</code> . Else → <code>persist_and_finish_node</code> .	Req 7

#	Node name	Type	What it does	Brief requirement
11	invoice_verification_node	Tool node (pandas)	Loads invoice_verification_records.csv, filters by customer_name, compares claimed vs. verified discrepancy within tolerance.	Req 7
12	router_verification	Conditional edge	If verified & matched → courtesy_resolution_node. If mismatch/not found → route to manual review (routed_team = 'Billing Department') then persist_and_finish_node.	Req 7
13	courtesy_resolution_node	Tool node — INTERRUPT BEFORE THIS NODE	Stages the resolution (credit amount, action text) into pending_action; graph pauses here for human approval.	Req 7 & 8
14	human_review_gate	External — Flask route, not a graph node	Manager sees classification, confidence, reasoning_trace, invoice tool output, and the pending action; clicks Approve / Reject / Override.	Req 8
15	resume_after_review_node	Tool node	Applies the manager's decision to state (approved → execute; rejected → cancel; override → apply corrected fields), then proceeds.	Req 8
16	persist_and_finish_node	Terminal	Writes the full audit row (see section 7 schema) to SQLite. End of graph for this request.	Req 9

4.1 Fixed routing matrix used inside route_team_node

classification_category	routed_team
Diagnostic & Appointment Request	Service Advisor / Dispatch
Warranty Coverage Dispute	Warranty Administrator
Parts Availability Complaint	Parts Department
Repair Quote Approval	Originating Service Advisor
Invoice & Billing Dispute (unresolved/unverifiable)	Service Manager / Billing

4.2 The self-healing retry loop (error recovery)

Any tool node can raise a Pydantic ValidationError (e.g. the LLM passed "\$19.75" instead of 19.75). Wrap tool calls in try/except; on failure, set state['error'] and retry_count += 1, and route back to the node that produced the bad arguments (usually classify_node) with the error message injected into the prompt so the LLM can self-correct. Cap retries at 2–3 and fall back to manual review if still failing — never loop forever.

4.3 Graph wiring (conceptual code)

```
from langgraph.graph import StateGraph, END
from langgraph.checkpoint.sqlite import SqliteSaver
```

```

builder = StateGraph(TriageState)

builder.add_node("classify", classify_node)
builder.add_node("context_retrieval", context_retrieval_node)
builder.add_node("duplicate_check", duplicate_check_node)
builder.add_node("auto_close", auto_close_node)
builder.add_node("prioritize", prioritize_node)
builder.add_node("route_team", route_team_node)
builder.add_node("invoice_verification", invoice_verification_node)
builder.add_node("courtesy_resolution", courtesy_resolution_node)
builder.add_node("resume_after_review", resume_after_review_node)
builder.add_node("persist_and_finish", persist_and_finish_node)

builder.set_entry_point("classify")

builder.add_conditional_edges("classify", route_by_confidence, {
    "needs_context": "context_retrieval",
    "confident": "duplicate_check",
})
builder.add_edge("context_retrieval", "classify") # loop back and re-classify with more context

builder.add_conditional_edges("duplicate_check", route_by_duplicate, {
    "duplicate": "auto_close",
    "unique": "prioritize",
})
builder.add_edge("auto_close", END)

builder.add_edge("prioritize", "route_team")
builder.add_conditional_edges("route_team", route_by_dispute_type, {
    "needs_invoice_check": "invoice_verification",
    "no_check_needed": "persist_and_finish",
})
builder.add_conditional_edges("invoice_verification", route_by_verification, {
    "verified": "courtesy_resolution",
    "unverifiable": "persist_and_finish",
})
builder.add_edge("courtesy_resolution", "resume_after_review")
builder.add_edge("resume_after_review", "persist_and_finish")
builder.add_edge("persist_and_finish", END)

checkpointer = SqliteSaver.from_conn_string("redline.db")
graph = builder.compile(
    checkpointer=checkpointer,
    interrupt_before=["courtesy_resolution"], # <-- the HITL approval gate
)

```

Important nuance: interrupt_before=["courtesy_resolution"] means the graph stops right before staging the resolution and waits. Your Flask layer detects the paused state (LangGraph tells you via graph.get_state(config)), shows the manager the dashboard, and only re-invokes the graph once a decision is made. Nothing is 'executed' until that happens — this is what keeps the system safely autonomous-but-supervised.

5. Recommended Project Structure

```
redline-triage/
├── backend/
│   ├── app.py                # Flask app factory + route registration
│   ├── config.py             # Reads/writes local config (API url/key/model) – see section 6
│   ├── config.json           # Local file storing current LLM settings (gitignored, has a .example)
│   ├── config.json.example   # Template so judges know what to fill in
│   └── graph/
│       ├── state.py          # TriageState TypedDict
│       ├── nodes.py          # all node functions (Section 4 table)
│       ├── tools.py          # duplicate check, invoice verification, context retrieval, routing matrix
│       ├── llm.py            # builds the LLM client from current config.json, structured-output schemas
│       └── build_graph.py     # StateGraph wiring + compile() + checkpointer
│   └── db/
│       ├── schema.sql        # CREATE TABLE statements (Section 7)
│       ├── database.py       # sqlite3 connection helper + CRUD functions
│       └── redline.db         # created at runtime (gitignored)
│   └── data/
│       ├── service_requests.csv
│       └── invoice_verification_records.csv
│   └── routes/
│       ├── requests_routes.py # GET /api/requests, GET /api/requests/<id>
│       ├── run_routes.py      # POST /api/requests/<id>/run (invoke the graph)
│       ├── review_routes.py   # POST /api/requests/<id>/approve|reject|override
│       ├── settings_routes.py # GET/POST /api/settings (model/API config)
│       └── stream_routes.py   # GET /api/requests/<id>/stream (SSE node-by-node progress, optional)
│   └── requirements.txt
├── frontend/
│   ├── index.html            # Dashboard (queue, priority, filters)
│   ├── request.html          # Single-request detail + reasoning trace + approve/reject/override
│   ├── settings.html         # Editable API URL / API key / model name / temperature
│   ├── graph-view.html       # Static + live-highlighted visual of the LangGraph node diagram
│   ├── css/style.css
│   └── js/
│       ├── api.js            # fetch() wrappers for every backend route
│       ├── dashboard.js
│       ├── request-detail.js
│       ├── settings.js
│       └── graph-view.js
└── README.md
```

6. Configurable API URL / API Key / Model (Editable From the UI)

Since this needs to run for different judges/testers with their own key (OpenAI, Groq, OpenRouter, or a fully local Ollama server with no key at all), never hardcode credentials. Use a small local JSON config file as the source of truth, exposed and editable through a Settings page in the UI.

6.1 backend/config.json.example

```
{
  "provider_label": "OpenAI-compatible",
  "base_url": "https://api.openai.com/v1",
  "api_key": "",
  "model": "gpt-4o-mini",
  "temperature": 0.1,
  "confidence_threshold": 0.80,
  "invoice_tolerance": 0.05
}
```

Copy this to `config.json` on first run (do this automatically in `app.py` if the real file doesn't exist yet, so the app never crashes on a fresh clone). Add `config.json` to `.gitignore` so nobody's real key ends up committed.

6.2 backend/config.py

```
import json, os, threading

CONFIG_PATH = os.path.join(os.path.dirname(__file__), "config.json")
_lock = threading.Lock()

def load_config() -> dict:
    with _lock:
        with open(CONFIG_PATH, "r") as f:
            return json.load(f)

def save_config(new_values: dict) -> dict:
    with _lock:
        current = load_config()
        current.update(new_values)
        with open(CONFIG_PATH, "w") as f:
            json.dump(current, f, indent=2)
        return current
```

Every place that builds an LLM client (`graph/llm.py`) calls `load_config()` fresh, so a settings change takes effect on the very next request with no server restart:

```
from openai import OpenAI
from backend.config import load_config

def get_llm_client():
    cfg = load_config()
    return OpenAI(base_url=cfg["base url"], api_key=cfg["api key"] or "not-needed", cfg["model"], cfg["temperature"])
```

For a fully offline/local model with Ollama: `base_url` = "`http://localhost:11434/v1`", `api_key` can be any placeholder string (Ollama ignores it), and `model` is whatever you've pulled, e.g. "`llama3.1`". The Settings UI should offer a one-click 'Use local Ollama' preset button that fills these fields in automatically.

6.3 Settings API routes

Method & path	Purpose
GET /api/settings	Returns current config (api_key masked, e.g. shows only last 4 chars).
POST /api/settings	Accepts base_url, api_key, model, temperature, confidence_threshold, invoice_tolerance; saves via save_config().
POST /api/settings/test	Sends a tiny 'ping' completion request with the given settings and returns success/failure — lets the user verify their key/URL/model works before running the full batch.

6.4 Settings page UI (settings.html)

- A form with fields: Provider label (free text), Base URL, API Key (password-style input with a show/hide toggle), Model name, Temperature slider, Confidence threshold slider, Invoice tolerance input.
- A 'Preset' dropdown: OpenAI / Groq / OpenRouter / Local Ollama / Custom — selecting one auto-fills base_url.
- A 'Test Connection' button wired to /api/settings/test with a green/red status pill.
- A 'Save' button wired to /api/settings; toast notification on success.
- The API key field never round-trips the real key back into the input on page load — show a masked placeholder like sk-••••1234 instead, and only overwrite it server-side if the user actually types a new value.

7. SQLite Schema

7.1 Audit / requests table (db/schema.sql)

```
CREATE TABLE IF NOT EXISTS requests (
  request_id          TEXT PRIMARY KEY,
  customer_name       TEXT NOT NULL,
  customer_tier       TEXT,
  submitted_at        TEXT,
  original_text       TEXT NOT NULL,
  status              TEXT DEFAULT 'new',      -- new | in_review | closed | closed_duplicate
  ai_classification   TEXT,
  confidence_score     REAL,
  extracted_entities_json TEXT,
  llm_reasoning_trace TEXT,                  -- newline-joined trace, or JSON array
  is_duplicate        INTEGER DEFAULT 0,
  duplicate_of_request_id TEXT,
  priority_score       INTEGER,
  routed_team         TEXT,
  claimed_discrepancy  REAL,
  verified_discrepancy REAL,
  invoice_verified     INTEGER,
  autonomous_action_staged TEXT,
  action_status        TEXT,                  -- staged | approved | rejected | overridden | executed | n/a
  human_override_flag  INTEGER DEFAULT 0,
  override_details    TEXT,
  created_at          TEXT DEFAULT CURRENT_TIMESTAMP,
  resolution_timestamp TEXT
);

CREATE INDEX IF NOT EXISTS idx_requests_customer ON requests(customer_name);
CREATE INDEX IF NOT EXISTS idx_requests_status   ON requests(status);
```

LangGraph's `SqliteSaver` manages its own internal checkpoint tables automatically inside the same `redline.db` file (or a separate `checkpoints.db` if you prefer to keep them apart) — you do not need to design those tables yourself; they are created by LangGraph on first use.

8. Frontend & UI Plan (Plain HTML/CSS/JS, Judge-Friendly)

8.1 Pages

Page	Purpose	Key elements
index.html — Dashboard	Main queue view	Sortable/filterable table of requests (priority desc by default), colored priority badges, category filter chips, a 'Run Batch' button that POSTs all 'new' rows through the graph one by one (or triggers an async job), Chart.js pie chart of category distribution and bar chart of priority buckets.
request.html — Detail view	One ticket, fully inspectable	Original text, classification + confidence badge, extracted entities as tags, full reasoning trace shown as a step timeline, duplicate-check result, invoice verification tool output (table row from the CSV if matched), routed team, and — if a courtesy resolution is pending — an Approve / Reject / Override panel.
graph-view.html — Agent visualization	Explains the architecture to judges	The static node/edge diagram (SVG, see 8.2) with the currently-active node highlighted in real time while a request is being processed (via SSE), plus a legend distinguishing LLM nodes, tool nodes, conditional edges, and the HITL gate.
settings.html — Configuration	Editable model/API config	See section 6.4.

8.2 Visualizing the LangGraph graph

You asked for a visual representation of the nodes/tools — two complementary options, both cheap to build:

- **Static diagram:** hand-author an SVG (or a Mermaid flowchart string rendered client-side via the Mermaid.js CDN script) that mirrors exactly the node table in Section 4 — boxes for each node, diamond shapes for conditional routers, a distinct color/icon for tool nodes vs. LLM nodes, and a dashed red border around the HITL approval gate.
- **Live highlight overlay:** have each node function emit a small event (e.g. push into a Python queue, or write a 'current_node' row to a lightweight table) that a Flask SSE route (/api/requests/<id>/stream) streams to the browser; graph-view.js listens and adds a CSS .active class (glow/pulse animation) to the matching SVG node id as the request moves through the graph. This is a great, low-effort 'wow' moment for a demo without needing a heavy visualization library.

```
<!-- Mermaid version — paste directly into graph-view.html and load the mermaid CDN script -->

```

```
CR --> HR[human_review_gate]::human --> RA[resume_after_review_node] --> F
classDef gate fill:#ffe3e3,stroke:#e94560,stroke-width:2px;
classDef human fill:#fff3e6,stroke:#8a3324,stroke-dasharray: 4 2;
</pre>
```

8.3 Visual style direction

- Dark navy / deep-blue header bar with a red-orange accent (matches an automotive 'diagnostic dashboard' feel) — e.g. background #16213e, accent #e94560, card backgrounds white/off-white, priority badges: red (urgent), amber (medium), green (low).
- Use a card-based layout with soft shadows for each ticket row rather than a plain dense HTML table — reads as more polished in a live demo.
- Sticky top navbar with three tabs: Dashboard · Agent Graph · Settings.
- Every AI decision surface should visibly show a confidence percentage and a small 'reasoning' expand/collapse — judges reward transparency into the model's thought process.

9. Flask API Surface (Full Endpoint List)

Method & Path	Description
POST /api/ingest	Loads service_requests.csv + invoice_verification_records.csv into the requests table (idempotent — skip rows already present).
GET /api/requests	List all requests with filters (?status=&category;=&sort;=priority).
GET /api/requests/	Full detail for one request, including reasoning_trace and tool outputs.
POST /api/requests//run	Invokes/resumes the LangGraph graph for this request_id (thread_id = request_id).
POST /api/requests/run-all	Runs the graph over every 'new' request sequentially (simple loop is fine for a competition demo).
GET /api/requests//stream	SSE stream of node-by-node progress for the live graph-view highlight (optional but impressive).
POST /api/requests//approve	Manager approves the pending courtesy resolution; resumes the graph.
POST /api/requests//reject	Manager rejects it; resumes the graph, cancels the action, still persists.
POST /api/requests//override	Manager supplies corrected fields (e.g. a different amount/category); injected into state before resuming.
GET /api/settings	Current model/API config (key masked).
POST /api/settings	Update model/API config.
POST /api/settings/test	Test the configured LLM connection.
GET /api/stats	Aggregate counts for dashboard charts (by category, by team, by override rate).

10. Phased Build Plan (Give This Order To Your AI Coding Assistant)

Phase 0 — Scaffolding

- Create the folder structure from Section 5.
- Set up a virtualenv, write requirements.txt: flask, flask-cors, langgraph, langchain-core, openai, pydantic, pandas, rapidfuzz, python-dotenv (optional).
- Copy the two provided CSVs into backend/data/.
- Write config.json.example and the config.py load/save helpers.

Phase 1 — Database + ingestion

- Write schema.sql and database.py (connect, init_db, insert/update/select helpers).
- Write POST /api/ingest to load both CSVs into the requests table (invoice CSV data is queried live at verification time via pandas — you don't need to duplicate it into SQLite unless convenient).

Phase 2 — LLM layer + structured classification

- Write graph/llm.py (client builder reading config.json).
- Define the Pydantic ClassificationResult schema (category literal, confidence_score, extracted_entities).
- Write classify_node using structured output / JSON schema mode.
- Unit-test it stand-alone (no graph yet) against a handful of rows from the CSV.

Phase 3 — Tools

- context_retrieval tool (SQLite query for related history).
- duplicate_check tool (rapidfuzz similarity within 48h window, same customer).
- invoice_verification tool (pandas filter + tolerance compare against invoice_verification_records.csv).
- prioritize logic + route_team fixed matrix.

Phase 4 — Build & compile the LangGraph graph

- Wire all nodes/edges per Section 4.3.
- Add the SqliteSaver checkpoint and the interrupt_before HITL gate.
- Test end-to-end on 3–4 representative rows: a clean diagnostic request, an obvious duplicate, a verifiable billing dispute, and an unverifiable billing dispute.

Phase 5 — Flask routes

- Wrap the graph in the run/approve/reject/override routes.
- Add the settings routes.
- Add /api/stats for the dashboard.

Phase 6 — Frontend

- Dashboard with table + charts.
- Request detail page with reasoning trace + approval panel.
- Settings page with presets + test-connection.
- Graph-view page with the Mermaid diagram + optional SSE live highlight.

Phase 7 — Polish for the demo

- Seed a 'Run Batch' button that processes all rows in one click for a live demo.
- Add empty/loading states so the UI never looks broken while the LLM is thinking.
- Add a short in-app 'How this works' modal summarizing the architecture, since judges appreciate a guided walkthrough.
- Write the MANIFEST.md and the rationale write-up required by the submission ('Your Solution' section of the brief) — this plan doubles as your source material for that write-up.

11. Recap: Feasibility, Risks & Honest Caveats

- **Fully possible:** everything described in this plan runs on a single local machine with Flask's built-in dev server; no cloud deployment, no Docker, no external DB required.
- **LLM choice is fully pluggable:** hosted API key (OpenAI/Groq/OpenRouter/etc.) or a fully offline model via Ollama — both work through the same OpenAI-compatible client code, switchable from the UI.
- **No real financial/external side effects** ever occur — 'issuing a courtesy resolution' means writing a row to your own SQLite database and rendering a message in your own UI. This satisfies the brief's 'autonomously issue a courtesy resolution via a tool call' requirement without touching real money, exactly matching your competition constraint.
- **One real limitation:** if you choose a small/local open-weights model (e.g. a 7–8B Ollama model) instead of a frontier hosted model, structured-output reliability and classification nuance will be noticeably weaker — budget extra time for prompt tuning and the retry loop if you go fully offline for the LLM as well as everything else. Using a hosted API key (even a free-tier one) for just the LLM calls, while keeping the DB/UI 100% local, is a good middle ground if allowed by your competition's rules.
- **SSE live-highlighting is optional** — if time is short, skip it and keep the static Mermaid diagram plus the reasoning-trace timeline on the request detail page; that alone comfortably satisfies the 'visual representation of nodes/tools' request.

This document is your build brief. Hand Sections 3–10 directly to an AI coding assistant (e.g. paste them into the assistant's first message along with the two CSV files) and ask it to implement one phase at a time, verifying each phase runs before moving to the next.